Software Requirements Specification

for

YouTube Video Intelligence MCP Server

Version 1.0

Prepared by Random Stuff

Aadyant Neog BT2024186
Jeswin Jose BT2024142
Paul T Mathew BT2024135

12 November 2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 3
2.6 User Documentation 3
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 4
3.1 User Interfaces 4
3.2 Hardware Interfaces 4
3.3 Software Interfaces 4
3.4 Communications Interfaces 4
4. System Features 4
4.1 Similar Videos 4
4.2 Similar Channels 5
4.3 Personalised Recommendations 5
4.4 Engagement Analysis 6
4.5 Constrained Playlist Creation 7
4.6 Authorisation and Account Management 8
4.7 Quota Budgeting 8
4.8 Input Validation Coverage and Errors 9
5. Other Nonfunctional Requirements 9
5.1 Performance Requirements 9
5.2 Safety Requirements 10
5.3 Security Requirements 10
5.4 Software Quality Attributes 11
5.5 Business Rules 11
6. Other Requirements 11
6.1 Agent Skill Requirements 11
6.2 Engineering and Knowledge Requirements 12
6.3 Verification and Traceability 13
Appendix A: Glossary 14
Appendix B: Analysis Models 15
Appendix C: To Be Determined List 16

Revision History

Name	Date	Reason For Changes	Version
Random Stuff	2026-11-07	Initial YouTube SRS and team roster.	1.0
Random Stuff	2026-11-09	Reorganised into the supplied IEEE template.	1.0
Random Stuff	2026-11-12	Condensed presentation; all numbered requirements retained.	1.0

1. Introduction

1.1 Purpose

This SRS defines version 1.0 of the YouTube Video Intelligence MCP Server for Team Random Stuff. It specifies required product behaviour, interfaces, quality, skills and acceptance evidence. The submitted topic and requirements form the fixed baseline; implementation and test results are separate deliverables.

1.2 Document Conventions

Shall denotes a mandatory obligation. FR, NFR, SK and GOV identify functional, quality, skill and engineering requirements; AT identifies acceptance scenarios. All numbered requirements and all system features have mandatory High priority unless a condition is stated. Requirement IDs remain stable.

1.3 Intended Audience and Reading Suggestions

Reviewers should read Sections 1-2 for scope, Sections 3-5 for interfaces and behaviour, Section 6 for skills, engineering and verification, and Appendices A-C for terminology, models and open items. Developers and testers shall use the numbered requirements and their verification links.

1.4 Product Scope

Learners need focused educational playlists without manually filtering distracting content, evaluating creators and checking runtimes. This local MCP server connects an AI client to the official YouTube Data API v3 for discovery, metadata-based similarity, personalised recommendations, collection analysis and constrained playlist creation after explicit human confirmation.

Excluded: scraping/unofficial APIs, video/audio downloads, arbitrary transcript retrieval, watch history, playback, uploads, comments, likes/subscription changes, and editing/deleting existing playlists. A hosted service, custom web application, model training and support for every AI client are not required. Summaries use available API metadata and identify that evidence basis.

1.5 References

Scope authorities are [R1] and [R2]. Technical references below were consulted on 7 November 2026. This document follows the teaching assistants' Karl E. Wiegers SRS template; its permission notice is retained.

[R1] YouTube Video Intelligence MCP Server project statement. Supplied Project_1_YouTube_MCP.pdf, pp. 1-2.

[R2] AI-Assisted SDLC. Supplied SDLC_and_Workflow.pdf, pp. 1-3.

[R3] Google. YouTube Data API reference. https://developers.google.com/youtube/data/reference/rest

[R4] Google. Videos list. https://developers.google.com/youtube/data/reference/rest/v3/videos/list

[R5] Google. PlaylistItems and Subscriptions resources. https://developers.google.com/youtube/data/reference/rest/v3/playlistItems ; https://developers.google.com/youtube/data/reference/rest/v3/subscriptions

[R6] Google. Quota calculator. https://developers.google.com/youtube/data/determine_quota_cost

[R7] Google. OAuth for desktop apps. https://developers.google.com/identity/protocols/oauth2/native-app

[R8] Google. OAuth token expiry. https://developers.google.com/identity/protocols/oauth2

[R9] Google. Playlist and item creation.
https://developers.google.com/youtube/data/reference/rest/v3/playlists/insert ;
https://developers.google.com/youtube/data/reference/rest/v3/playlistItems/insert

[R10] MCP specification 2025-11-25, tools and related sections.
https://modelcontextprotocol.io/specification/2025-11-25/server/tools

[R11] Agent Skills specification. https://agentskills.io/specification

[R12] Google. YouTube Developer Policies, III.E.4.
https://developers.google.com/youtube/terms/developer-policies

2. Overall Description

2.1 Product Perspective

The product is a new local integration used by an existing AI client. The server owns validation, metadata processing, quota/proposal state and write safeguards. Google/YouTube provide identity, consent, metadata and playlist operations. Appendix B shows the system boundary and lifecycle.

2.2 Product Functions

FR-1 Similar videos. Rank videos related to a supplied video URL or ID.

FR-2 Similar channels. Rank channels related to a channel URL, handle, or ID.

FR-3 Recommendations. Use authorised likes, subscriptions, and playlists as preference signals.

FR-4 Engagement analysis. Report top channels and categories, duration preferences, and supported time comparisons.

FR-5 Playlist creation. Interpret a request, propose a compliant playlist, obtain approval, and create it.

2.3 User Classes and Characteristics

Learner. Primary user. Supplies a seed/topic and constraints, grants Google consent, reads results and approves or rejects a proposed playlist. Technical API knowledge is not required.

Team developer. Configures local credentials and dependencies, maintains provider integrations, skills, tests and brain notes. Must understand code and documentation submitted under the team workflow.

Instructor or reviewer. Examines requirements and evidence, observes demonstrations and checks traceability. Does not receive the learner's credentials or unrestricted account access.

2.4 Operating Environment

The baseline shall run locally for one authorised account per server instance on a documented Windows development environment. The implementation shall include a reproducible install or build command and dependency versions. A supported desktop MCP client and a browser for Google consent are prerequisites. Internet connectivity, an enabled Google Cloud project, credentials, and a YouTube channel are required for live account operations. Fixture mode shall work without live credentials.

Performance tests use the NFR-08 reference machine: at least four CPU cores and 8 GB RAM. No special hardware or GPU is required; client/runtime and dependency versions are design selections.

2.5 Design and Implementation Constraints

Use only YouTube Data API v3 through a replaceable provider. Compute similarity locally from metadata; assume neither watch history nor related-video/channel endpoints. CI uses fixtures; extra Cloud projects shall not evade quotas. Only approved new-playlist creation and item insertion are in scope.

2.6 User Documentation

Deliver versioned Markdown/client help for setup, connection, consent/privacy, feature examples, approval, limits and recovery. Developer documentation covers MCP schemas/resources, provider interfaces and tests. GOV-04 governs OpenAPI documentation only if a REST interface is introduced.

2.7 Assumptions and Dependencies

A-01 Client boundary. At least one supported client can load skill instructions, display structured results and provide trusted confirmation. A documented local approval path shall be used if the client cannot provide trustworthy approval.

A-02 API data limits. Only accessible, budget-bounded metadata is available. Unknown language/duration, missing tags, API result caps and absent historical events reduce coverage and shall be disclosed.

A-03 Consent and channel. The user supplies consent, configured test-user access and a YouTube channel for creation. Denial, Testing-mode expiry or missing channel shall produce guided recovery.

D-01 Quota changes. The handout describes the 10,000-unit default. Current official documentation additionally separates search into its own daily allowance. The implementation shall use a dated configurable quota profile rather than a permanently hard-coded assumption.

D-02 API policy review. The assignment requires computed similarity and aggregated analysis. YouTube Developer Policies Section III E 4 restricts derived data and metrics. The team shall resolve the applicability of those restrictions with the instructor and, where needed, the API provider before enabling affected live processing or public deployment. Labelling an application score is not itself evidence of permission.

R-01 Data and content quality. Metadata can be incomplete, misleading or changed after preview. Show its evidence and retrieval time; revalidate before creation. Do not claim verified educational quality.

R-02 Partial remote writes. YouTube operations are separate API calls, so playlist creation and insertion are not one atomic transaction. Preserve operation state, disclose partial results and reconcile uncertain outcomes.

R-03 Evaluation evidence. Use fixtures to verify required algorithms and error paths while external dependencies are unavailable. Keep fixture and live evidence distinct. An unresolved live dependency remains a block to the affected live acceptance scenario.

Appendix C tracks the unresolved policy and design-selection items. They do not silently remove core features or establish that live acceptance has passed.

3. External Interface Requirements

3.1 User Interfaces

The AI client accepts identifiers or natural-language goals and displays source links, durations, reasons, errors and the exact playlist proposal. Approval uses a trusted client control or separate local human interaction. Client-specific screen design is documented during implementation.

3.2 Hardware Interfaces

Use standard host process, disk, protected credential storage, browser and network facilities. No sensors, actuators, specialist peripherals or direct device protocols are required.

3.3 Software Interfaces

NFR-01 MCP interoperability. The server shall support the standard local stdio transport, negotiated protocol initialisation, tool discovery and invocation, and resource discovery and reading. Tool inputs shall have machine-readable schemas; outputs shall include machine-readable fields and a readable explanation. Diagnostic logging shall use stderr and shall not corrupt stdout protocol messages. At least one documented MCP client shall be demonstrated.
Verification: AT-01 and AT-19; [R10].

Conversational client. Accept natural-language requests; display ranked results, warnings, proposals and approval controls.

Google OAuth. Browser-based installed-app authorisation with a loopback callback, state validation and PKCE. Tokens do not pass through chat.

YouTube Data API v3. Retrieve video, channel, subscription and playlist metadata; create only an approved new playlist and its items.

MCP resources. Expose non-secret capability/help text, current quota status and account-scoped proposal/report resources.

Agent skills. Ship versioned SKILL.md workflows. Make instructions discoverable as resources and loadable by the selected skill-capable client.

The provider exchanges validated IDs and structured metadata; tokens stay in protected local storage. Proposals, quota and operation records are account-scoped. SKILL.md workflows are instructions loaded by the selected client, not a new MCP primitive.

3.4 Communications Interfaces

The client launches the local MCP process and exchanges structured messages over stdio; diagnostics use stderr. Google OAuth/API traffic uses HTTPS and the consent callback uses loopback only. No public listener, email or FTP service is required. See FR-8 and NFR-02/07/09.

4. System Features

All features below are mandatory. The concise response sequences supplement the binding numbered requirements; shared error handling applies throughout.

4.1 Similar Videos

4.1.1 Description and Priority

Priority: High. Given a seed video, identify distinct videos with explainable metadata overlap.

4.1.2 Stimulus/Response Sequences

Input video URL/ID and limit -> validate and reserve quota -> retrieve seed/candidates -> rank distinct videos with reasons, or return a typed availability/data error.

4.1.3 Functional Requirements

FR-1.1 Retrieve a seed and candidate pool. Given a valid video URL or ID, the server shall retrieve its accessible metadata and discover a bounded pool of candidates through official API searches or channel metadata. An unavailable seed shall produce a typed error. Candidate retrieval shall not assume a related-videos endpoint exists.
Verification: AT-02 and AT-10; [R1, R3, R4].

FR-1.2 Compute and explain similarity. The server shall compute video similarity locally from available titles, descriptions, tags and categories using an explainable technique such as TF-IDF with cosine similarity or documented weighted scoring. It shall return at most the requested number of unique candidates in descending score order, exclude the seed, and break ties by a documented stable rule. Each result shall include its URL, title, channel, duration, application score and short metadata-based reason.
Verification: AT-02; unit tests for scores, ordering and ties.

FR-1.3 Handle incomplete evidence. Missing tags or descriptions shall not crash ranking. The server shall use remaining supported features and disclose reduced evidence. It shall return an empty result with a reason when no eligible candidates are available. Results shall not describe similarity as verified educational quality or as a recommendation issued by YouTube.
Verification: AT-02 and AT-10.

4.2 Similar Channels

4.2.1 Description and Priority

Priority: High. Resolve a channel and compare its metadata and bounded video sample with other channels.

4.2.2 Stimulus/Response Sequences

Input channel URL/handle/ID -> resolve and compare bounded metadata samples -> return distinct ranked channels with evidence and limitations.

4.2.3 Functional Requirements

FR-2.1 Resolve and compare channels. Given a channel URL, @handle or ID, the server shall resolve the seed channel and rank distinct candidate channels by available channel descriptions and a bounded sample of their video metadata. The response shall disclose the sample size and selection rule. It shall exclude the seed channel and shall not rely on an assumed related-channels endpoint.
Verification: AT-03; [R1, R3].

FR-2.2 Return explainable channel rankings. Each channel result shall include ID, title, URL, application score and a short explanation of the topic overlap. Identical inputs, fixtures and configuration shall produce identical ordering. Unresolvable handles, inaccessible channels or insufficient comparable metadata shall return an explicit error or insufficient-data result.
Verification: AT-03 and AT-10.

4.3 Personalised Recommendations

4.3.1 Description and Priority

Priority: High. Use accessible, authorised likes, subscriptions and playlists as preference evidence.

4.3.2 Stimulus/Response Sequences

Grant read consent -> retrieve accessible likes/subscriptions/playlists -> return explained recommendations and coverage; request a seed if no usable signals exist.

4.3.3 Functional Requirements

FR-3.1 Read authorised preference signals. With read consent, the server shall retrieve accessible liked videos, subscriptions, and playlists and their accessible items. It shall form a preference profile from the available sources, deduplicate video IDs across collections, and identify each source and its coverage. It shall not obtain or infer an actual watch-history feed.
Verification: AT-04; [R1, R4].

FR-3.2 Recommend and explain. The server shall discover and rank a bounded set of videos using that profile and any explicit topic, duration or language filters. Each result shall identify the signals supporting the recommendation. Results shall exclude seed/profile videos by default and shall disclose this exclusion. The same fixtures and settings shall produce a stable ranking.
Verification: AT-04 and AT-15.

FR-3.3 Support sparse and denied data. If a source is empty or inaccessible, the server shall report the limitation and use remaining consented sources. If no usable preference data exists, it shall request a seed video or topic rather than invent preferences. Permission errors shall never be bypassed by scraping or another account.
Verification: AT-04 and AT-11.

4.4 Engagement Analysis

4.4.1 Description and Priority

Priority: High. Summarise accessible collection activity and compare supported time periods without claiming watch history.

4.4.2 Stimulus/Response Sequences

Select collections and periods -> use supported event timestamps -> return counts, duration statistics and valid comparisons with denominators and missing-data limits.

4.4.3 Functional Requirements

FR-4.1 Summarise accessible collections. The server shall report the top channels and video categories by item count, duration preferences through count, mean and median video length, and counts in documented duration bands. It shall distinguish liked videos, subscriptions and playlist items, identify the denominator of each count or percentage, and state duplicate handling, missing metadata and the retrieved population. Subscription summaries shall count channels rather than pretend subscriptions are video views.
Verification: AT-05 with hand-calculated fixtures.

FR-4.2 Compare periods with valid timestamps. The server shall support two user-specified non-overlapping periods for trends and comparisons using available playlist-addition and subscription-creation timestamps. UTC intervals shall be start-inclusive and end-exclusive. Reports shall name the timestamp basis and include absolute differences; percentage change from a zero baseline shall be marked undefined. Video publication dates shall not be substituted for engagement dates. Unsupported like-event dates shall be reported as unavailable.
Verification: AT-05, including boundaries, zero baselines and missing dates; [R5].

FR-4.3 Avoid unsupported conclusions. Reports shall label the data as accessible collection activity and shall not claim watch time, completion, full historical behaviour or learning progress. A video runtime describes the video, not time actually watched. Empty periods shall return zero supported counts and a clear insufficient-evidence explanation for other measures.
Verification: AT-05; [R1].

4.5 Constrained Playlist Creation

4.5.1 Description and Priority

Priority: High. Interpret a playlist request, show a feasible proposal, obtain exact human approval and create the playlist safely.

4.5.2 Stimulus/Response Sequences

Specify constraints -> preview a compliant proposal -> reject, revise or approve the exact version -> recheck and create -> return the URL and actual completed/partial/unknown state. Rejection makes no writes.

4.5.3 Functional Requirements

FR-5.1 Translate and validate a request. The AI client and server together shall accept a natural-language playlist request specifying topic, total duration, maximum video length and language. The client shall translate these into structured values; the server shall independently validate them. Missing or ambiguous hard constraints shall be clarified before proposing a playlist. A request for an exact total shall be clarified as an upper limit or rejected as unsupported, rather than silently reinterpreted.
Verification: AT-06 and AT-10.

FR-5.2 Enforce hard constraints. The server shall build an ordered proposal whose total duration does not exceed the requested total and whose individual durations do not exceed the requested maximum. Every selected video shall satisfy the stated topic-selection rule and requested language according to available API language metadata. Unknown language shall not count as a strict match, and search relevanceLanguage alone shall not prove spoken language. Unverifiable candidates shall be excluded and the evidence limitation shall be shown.
Verification: AT-06, AT-07 and duration-boundary unit tests.

FR-5.3 Handle insufficient candidates. The server shall report if no non-empty feasible proposal can be built. If only a shorter compliant playlist is available, it shall show the total and unused time and allow the user to accept it or revise the request. It shall never relax a hard constraint, add duplicate filler or claim exact duration without evidence.
Verification: AT-07.

FR-5.4 Present the complete proposal. Before any playlist write, the client shall display the destination account/channel, title, privacy, topic and constraints, ordered video titles and links, channels, per-video duration, total duration, language evidence, warnings and proposal expiry. Default privacy shall be private. The proposal shall have an account-bound ID and version tied to the exact content. Any change to account, videos, order, title, privacy or constraints shall invalidate earlier approval.
Verification: AT-06, AT-08 and AT-12.

FR-5.5 Require trusted human approval. The write operation shall require explicit human approval of the current proposal through a trusted client confirmation control or a local approval interaction outside model-controlled tool arguments. Approval shall be bound to the user/account, proposal version and operation, expire within 10 minutes, and be usable for only that operation. A supplied confirmed=true value or instructions embedded in metadata shall not establish approval. Rejected, missing, expired, mismatched or superseded approval shall cause zero playlist writes.
Verification: AT-08 and AT-12.

FR-5.6 Create and report the playlist. After valid approval, the server shall verify write scope, destination channel, current feasibility and quota, then create the new playlist and insert exactly the approved videos in the approved order. It shall return the playlist URL, final item count and duration, and a success or partial-failure status. It shall not silently replace an unavailable video or change privacy. A material change found during revalidation shall require a new preview and approval.
Verification: AT-06, AT-09 and AT-11; [R9].

FR-5.7 Prevent duplicate writes and disclose partial state. The server shall record the operation and known remote IDs. Repeating a completed operation ID shall return its recorded result without another playlist creation. On an insertion failure it shall stop and report the playlist URL, inserted and pending items. An ambiguous network timeout on a write shall be reconciled before any retry; if unresolved, it shall report an unknown outcome and require user action. Resume shall target only verified missing items, with renewed approval if needed. Automatic deletion is outside the baseline.
Verification: AT-09.

Example: a two-hour limit is at most 7,200 seconds, with each item at most 1,200 seconds for a 20-minute cap. A 1,201-second item fails. A 6,420-second proposal reports 107 minutes used and 13 unused; unknown metadata cannot prove compliance.

4.6 Authorisation and Account Management

4.6.1 Description and Priority

Priority: High. Establish delegated account access and recover from consent expiry, missing scope or missing YouTube channel.

4.6.2 Stimulus/Response Sequences

Request private data/write access -> browser consent -> protect/refresh tokens. On expiry or revocation, guide re-authorisation; disconnect clears local account data and attempts revocation.

4.6.3 Functional Requirements

FR-6 Manage consent and account state. The server shall support authorisation, access-token refresh, re-authorisation after expiry or revocation, and disconnect. Read workflows shall request youtube.readonly; creation shall request a documented sufficient write scope such as youtube.force-ssl only when needed. The client shall explain scope purpose. Missing channel or youtubeSignupRequired shall return guidance without claiming creation succeeded. Disconnect shall remove local tokens and account-specific stored data and attempt token revocation.
Verification: AT-11 and AT-14; [R7, R8, R9].

4.7 Quota Budgeting

4.7.1 Description and Priority

Priority: High. Reject unaffordable workflows before provider traffic and account for bounded calls safely.

4.7.2 Stimulus/Response Sequences

Request a workflow -> estimate/reserve all applicable buckets -> reject before calls if insufficient -> account for attempts and release unused reservations.

4.7.3 Functional Requirements

FR-7 Preflight and account for quota. Before the first provider call in a workflow, the server shall estimate and reserve its bounded worst-case cost across all applicable quota buckets, including pagination, revalidation, writes and permitted retries. If any bucket is insufficient, it shall reject the workflow before any provider call. It shall track each attempted call, including errors, and release unused reservations. Parallel operations shall not oversubscribe the tracked balance.
Verification: AT-13 and AT-15; [R6].

4.8 Input Validation Coverage and Errors

4.8.1 Description and Priority

Priority: High. Apply shared input and coverage rules and return safe, actionable error states.

4.8.2 Stimulus/Response Sequences

Receive inputs -> validate/normalise before calls -> disclose pagination and inaccessible data -> return stable, secret-free errors with safe next actions.

4.8.3 Functional Requirements

FR-0.1 Normalise and validate inputs. The server shall accept canonical YouTube video URLs, youtu.be links and video IDs, and channel URLs, @handles and channel IDs. It shall validate identifiers, positive durations, result limits, language codes, date ranges, title and privacy fields before provider calls. Unknown hosts or ambiguous legacy channel URLs shall return corrective guidance instead of being fetched as arbitrary URLs.
Verification: AT-02, AT-03 and AT-10.

FR-0.2 Declare coverage and limits. Every collection result shall state whether it is complete, paginated, sampled or truncated and shall report inaccessible items. The baseline shall support result limits from 1 to 50, a configurable candidate cap with a default of 200 unique videos, and proposals containing 1 to 50 videos. Larger requests shall return a clear limit message. Library pagination shall stop at the configured request budget and report the continuation state; it shall not label partial data as a full account history.
Verification: AT-04 and AT-15.

FR-8 Return recoverable errors. The server shall return stable error codes, a readable cause, retryability and a safe next action for invalid input, unavailable content, insufficient data, consent failure, quota exhaustion, rate limiting and provider failures. Errors shall not leak credentials, private metadata or stack traces. Partial results shall be explicitly marked.
Verification: AT-10, AT-11 and AT-16.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

NFR-08 Meet local performance targets. On a documented reference machine with at least four CPU cores and 8 GB RAM, fixed offline fixtures and no client-model generation time, input validation shall have a 95th-percentile latency of at most 100 ms; ranking up to 200 candidate videos or constructing a proposal shall take at most 2 seconds. Tests shall use at least 30 measured runs after one warm-up. Cached read responses shall have a 95th percentile of at most 2 seconds. Live network and OAuth latency shall be reported separately.
Verification: AT-17 benchmark report with hardware, inputs and timings.

NFR-09 Bound runtime and resource use. A provider HTTP attempt shall have a configurable timeout with a default of 15 seconds. A foreground read or planning workflow shall stop after a configurable deadline with a default of 60 seconds, return a typed timeout or labelled partial result, and perform no writes. Creation shall report progress and honour the same per-attempt timeout; a stop shall preserve partial/unknown state. All searches and pagination shall respect the declared candidate and quota caps.
Verification: AT-15 and AT-16 with simulated delays and bounded-call assertions.

NFR-12 Track changing API quotas. Quota method costs, bucket limits and daily reset rules shall be configurable and versioned with an authoritative reference date. The initial profile shall account for a 10,000-unit daily general bucket and the separately documented 100-call daily search allowance. The local ledger shall survive server restart and use the provider's reset boundary. It shall expose estimated remaining budget and warn that other applications sharing the Cloud project can consume unobserved quota.
Verification: AT-13 and AT-15; [R6].

Timing targets apply to local fixtures. Report model, network and OAuth latency separately. Quota configuration remains dated and changeable.

5.2 Safety Requirements

There is no physical equipment control or claimed safety certification. Account/data safeguards are explicit approval (FR-5.5), revalidation (FR-5.6), duplicate/partial-write recovery (FR-5.7, NFR-06) and privacy controls. Metadata relevance does not guarantee factual or teaching quality.

5.3 Security Requirements

NFR-02 Protect credentials and transport. All external requests shall use HTTPS. OAuth access and refresh tokens shall be stored through OS-protected credential storage or an equivalently protected encrypted local store, with the key outside the repository. Secrets shall never appear in source, fixtures, brain notes, chat, logs, resources or tool results. Automated secret scanning shall run on pull requests.
Verification: AT-14; inspect repository, logs and token storage.

NFR-03 Enforce authority boundaries. The server shall validate schemas and account bindings on every tool call and resource read. It shall treat titles, descriptions, tags and retrieved text as data, never instructions. Only the trusted human approval path shall authorise playlist writes; the model and external metadata shall not be able to mint approval. Cross-account or modified-proposal access shall fail.
Verification: AT-12 and AT-14 adversarial tests.

NFR-04 Minimise personal data. Only data needed for the requested workflow shall be collected. Private account data shall remain isolated from other accounts and shall not be sent to the AI client except as needed for the requested result. Fixtures and CI logs shall contain synthetic or fully anonymised data. A user-facing privacy notice shall explain collection, AI-client disclosure, retention, disconnect and local deletion.
Verification: AT-14 and documentation inspection.

NFR-05 Limit retention and support removal. API metadata caches shall expire within 24 hours by default and shall be refreshed or deleted no later than 30 days. Local proposal and operation records containing API data shall follow that limit. Disconnect or a user deletion request shall clear account-specific caches, proposals, operation data and tokens during successful completion. Failed removal shall be reported and shall not be marked complete. Data deletion in this application shall not claim to delete the user's YouTube content.
Verification: AT-14 with simulated time and deletion checks; [R12].

NFR-07 Constrain network destinations. The server shall parse supported YouTube identifiers locally and shall not fetch arbitrary user-supplied URLs. Provider traffic shall be limited to official Google OAuth and YouTube API destinations required by the workflow. The OAuth callback shall bind only to the local loopback interface. The baseline shall not expose a public network listener.
Verification: AT-10 and AT-14; inspect outbound requests and callback binding.

5.4 Software Quality Attributes

Correct constraints, human approval and privacy take precedence over ranking breadth and latency. Required attributes are reliability, usability, maintainability and repeatable testing; interoperability is defined in NFR-01.

NFR-06 Fail safely and recover. Read requests may use at most three attempts with bounded exponential backoff and jitter for retryable failures. Quota exhaustion and permanent permission errors shall not trigger repeated calls. Write timeouts shall follow FR-5.7. After interruption, stored operation state shall distinguish not-started, in-progress, completed, partial and unknown outcomes. A failed write shall never be reported as complete.
Verification: AT-09 and AT-16 fault-injection tests.

NFR-10 Provide usable and explainable results. The client shall display human-readable durations, clickable source URLs, collection coverage, relevant constraints and actionable errors. Scores shall be labelled application estimates and shall identify the metadata used. Confirmation shall explicitly identify account and privacy. User and developer documentation shall include setup, consent, each feature, limits, recovery and one complete example.
Verification: AT-19 task walkthrough and documentation review.

NFR-11 Support maintainable testing. YouTube access shall be isolated behind a provider interface replaceable by a fixture provider. Domain rules for similarity, duration parsing, constraints and approval shall be testable independently of a live API or AI model. Dependencies and the supported runtime shall be declared. CI shall build or validate the package, lint, test, scan secrets and validate skills and brain notes on every PR.
Verification: AT-18 and AT-20; [R2].

5.5 Business Rules

Only the authorised learner can approve a specific current playlist proposal; OAuth alone is not that approval. FR-5.4 to FR-5.6; FR-6

A total-duration request is an upper bound. Do not fill unused time by violating language, topic, duration or uniqueness constraints. FR-5.1 to FR-5.3

Writes create a new playlist and its approved items only; default privacy is private. FR-5.4; FR-5.6; scope in Section 1.4

Untrusted metadata cannot grant authority or alter budget rules. FR-5.5; FR-7; NFR-03

Retain the submitted topic and requirement baseline; changes need instructor direction. GOV-05

6. Other Requirements

6.1 Agent Skill Requirements

Each skill is a versioned SKILL.md directory with valid name/description frontmatter, inputs, invocation conditions, workflow, outputs and failure handling. It cites a distilled note and cannot override server checks or approval.

SK-01 brain ingest. The brain-ingest skill shall capture supplied material or research as source-linked notes, record authorship and date, add requirement IDs and related wikilinks, update the index, and append the run to the log. Stage 1 shall contain at least 10 distinct research notes created through this workflow. Unverified material shall be identified as such.
Verification: AT-18: ingest sample material; inspect at least 10 source-linked research notes.

SK-02 brain refine. The brain-refine skill shall organise notes into PARA, merge duplicates while preserving sources and authorship, and repair or report broken links. It shall report the changes made and leave an auditable history through the normal PR workflow.
Verification: AT-18: duplicate-note and broken-link fixtures.

SK-03 brain query. The brain-query skill shall answer project questions or draft a document from relevant notes, cite the notes supporting its claims, and disclose missing or conflicting evidence. It shall not invent decisions, approvals, test results or sources.
Verification: AT-18 and AT-19: answer a requirement-to-decision question and an unsupported question.

SK-04 local check. The local-check skill shall build or validate the project, run the automated test suite, start the local server, invoke one read-only fetch tool through MCP, and report pass, fail or blocked separately for every step. It shall terminate resources it started. Fixture mode shall be the default; an explicitly configured live check shall report quota used.
Verification: AT-19: live demonstration of all five steps with individual statuses.

SK-05 video to playlist. The video-to-playlist skill shall accept a seed video, summarise its available title/description metadata with a stated evidence basis, find similar videos through FR-1, collect playlist constraints, propose a playlist, obtain human confirmation, create it through FR-5, and present a short analysis of duration, channels and constraints. Rejection or an infeasible request shall end without writes. It shall not claim access to full video content or unavailable transcripts.
Verification: AT-06, AT-08 and AT-19; full accepted and rejected workflow evaluations.

SK-06 Evaluate skills before submissions. Each applicable skill shall have repeatable evaluation cases covering a successful path, a negative path and evidence/authority handling. Evaluations shall run before each relevant stage submission and results shall be recorded with inputs, expected behaviour, observed behaviour and pass/fail/blocked status.
Verification: AT-18; stage-specific evaluation records, not generated claims of success.

6.2 Engineering and Knowledge Requirements

GOV-01 Maintain linked project knowledge. The repository shall contain an Obsidian-compatible brain vault with inbox, projects, areas, resources and archive directories, an index entry point and an append-only run log. Notes shall link to other notes, requirement IDs and real issues/PRs as available. Every team member shall author notes under their own identity.
Verification: AT-20; repository and commit-history review; [R2] SB-1, SB-6.

GOV-02 Record decisions and AI work. The repository shall contain root AGENTS.md and significant design decisions in docs/adr/. Each ADR and skill shall cite at least one distilled note. Significant AI sessions shall record goal, outcome and what worked or failed. The team shall write a review note at each stage and shall be able to explain every merged artefact.
Verification: AT-20; [R2] SB-2 to SB-4 and AI policy.

GOV-03 Use reviewed version control. All work and defects shall be tracked as GitHub Issues. Code, skills, brain and document changes shall use meaningful incremental commits and PRs with at least one human review. The team shall document its branching strategy and maintain release tags and a changelog. Contributions shall remain attributable to their actual authors.
Verification: AT-20; inspect issues, commit history, PR reviews and release records; [R2].

GOV-04 Maintain engineering artefacts. The team shall maintain architecture and design documentation with suitable UML/ER diagrams, ADRs, a development plan and task breakdown, user documentation and developer documentation for MCP tools/resources and provider interfaces. An OpenAPI document is required only if a REST interface is introduced.
Verification: AT-20; [R2] SE-2 to SE-4 and SE-7.

GOV-05 Preserve traceability and the submitted baseline. The team shall maintain requirement-to-issue-to-PR-to-test-to-skill-to-brain-note traceability. Before submission, requirement edits shall be reviewed. After submission, the selected topic and SRS shall remain the fixed baseline; implementation decisions shall refine it without silently changing scope. Instructor-directed clarifications shall be recorded separately.
Verification: AT-20; traceability completeness and baseline comparison.

1 Requirements. SRS with FR/NFR/SK; PARA vault, index/log, at least 10 ingested research notes; AGENTS.md; repository and CI skeleton.

2 Design. Architecture, diagrams and ADRs; brain-refine and brain-query; initial local-check.

3 Testing and CI. Continuous testing/CI plan and results; evaluated video-to-playlist; defect log and regression tests; stage reviews.

4 Review. Live FR-1 to FR-5 demonstration; product skill, local-check and brain-query; traced requirement walkthrough; final release tag.

6.3 Verification and Traceability

The acceptance catalogue below defines planned evidence. Requirement-level verification links identify the applicable cases; product test execution is not claimed.

ID	Acceptance focus	ID	Acceptance focus
AT-01	MCP connection, schemas, resources and stdout integrity.	AT-11	Expired/revoked tokens, failed refresh, denied scopes, missing channel.
AT-02	Video rankings, seed/duplicate exclusion, ties and missing tags.	AT-12	Metadata injection and model-forged approval are rejected.
AT-03	Channel resolution and explainable distinct rankings.	AT-13	Bucket preflight before any call and concurrent reservations.
AT-04	All preference sources, sparse data and pagination.	AT-14	Least privilege, credentials, isolation, redaction, retention and deletion.
AT-05	Analysis arithmetic, period boundaries and unavailable dates.	AT-15	Caps, pagination, quota accounting, reset and restart.
AT-06	Constrained preview, approval and exact creation workflow.	AT-16	Rate limits, timeouts, provider faults and bounded retries.
AT-07	Constraint boundaries, unknown metadata and infeasibility.	AT-17	NFR-08 offline performance benchmark.
AT-08	Missing, declined, expired, changed, replayed or wrong-account approval; no writes.	AT-18	Skill evaluations, note/link/schema validation and offline CI.
AT-09	Partial writes, restart, timeout reconciliation and replay.	AT-19	Documentation, selected client and live skill walkthroughs.
AT-10	Malformed inputs, unsupported hosts and unavailable content.	AT-20	Design, plan, ADR, review, traceability and release evidence.

TQ-1 Unit. Test domain functions, services and tool handlers, especially similarity, ISO 8601 duration parsing and constraint boundaries.

TQ-2 Integration. Test MCP handlers, services, state storage and interchangeable YouTube providers together.

TQ-3 System. Run AT-02 to AT-06 through the MCP interface for all five core features.

TQ-4 Negative. Cover invalid IDs, private/deleted videos, missing tags, quotaExceeded, youtubeSignupRequired, expired tokens and rate limits.

TQ-5 CI and live separation. PR CI uses synthetic fixtures/mocks with no live credentials. A separate scheduled live workflow has a fixed, configurable quota budget and authorised test account.

TQ-6 Quota. Test full-workflow budget preflight before any API call, accounting for both current bucket profiles and configured cost changes.

TQ-7 Skills. Evaluate applicable skills before every stage submission and retain the results.

TQ-8 Regression. Maintain tests as the product changes and give every significant defect a regression case.

Acceptance requires passing evidence for every applicable mandatory requirement. Record failures and external blocks explicitly. Approval bypass, secret disclosure, cross-account access or duplicate writes block release. Significant defects require regression tests; blocked cases are never marked passed.

GOV-05 requires the maintained requirement-to-issue-to-PR-to-test-to-skill-to-note matrix. Add actual links as work is created; missing links remain pending. Requirement IDs and AT references in this SRS provide the baseline mapping.

Appendix A: Glossary

Term	Definition
MCP	Model Context Protocol; the interface by which a client discovers and calls server tools and reads resources.
SRS	Software Requirements Specification; the project's required behaviour and constraints.
OAuth	Delegated authorisation to an account; separate from confirmation of a particular playlist.
TF-IDF	Term-frequency inverse-document-frequency; a text feature representation for similarity.
Cosine similarity	Comparison of the directions of two feature vectors.
ADR	Architecture Decision Record, linked to supporting knowledge notes.
CI	Continuous integration checks run on proposed repository changes.
PARA	Projects, Areas, Resources and Archive; the organising system for the Second Brain.
Fixture	Synthetic or fully sanitised test data used without live account access.
Proposal	An account-bound read-only playlist plan that still requires human approval.

Term	Definition
Quota bucket	A provider-defined allowance against which particular API methods are accounted.
TBD	A tracked item awaiting a decision; it does not make mandatory features optional.

Appendix B: Analysis Models

These conceptual models show the boundary and required playlist lifecycle; they do not prescribe implementation technology.

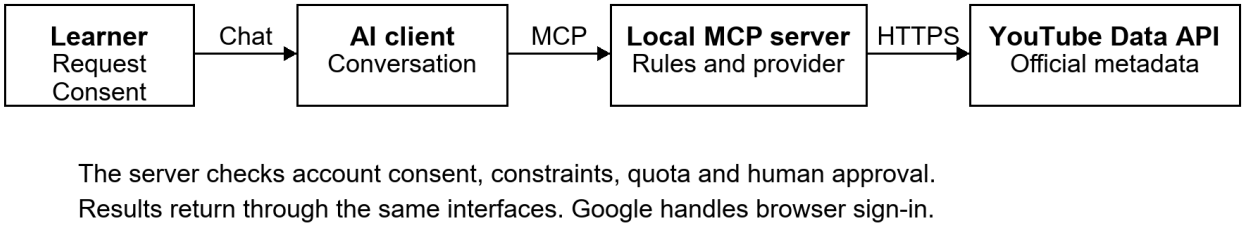


Figure B1 System context

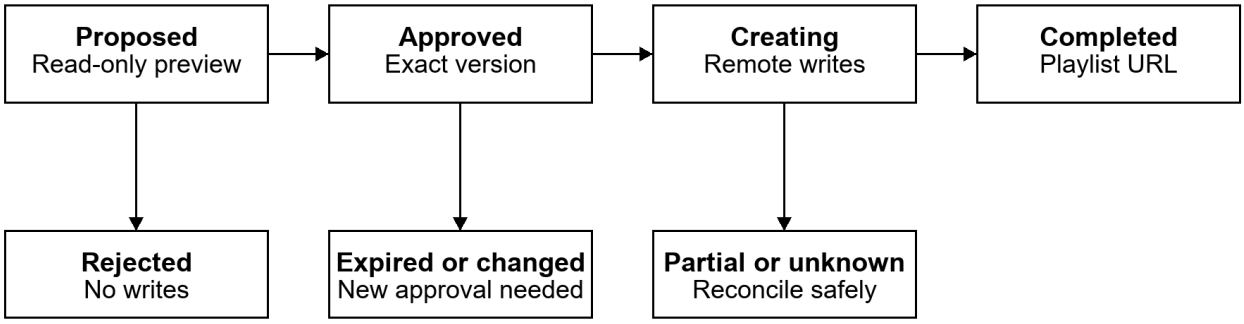


Figure B2 Playlist approval and execution lifecycle

Changed/expired proposals need fresh approval. Revalidate before writes, reconcile uncertain outcomes and return recorded results on replay (FR-5.4 to FR-5.7).

Core information records use the following fields; unknown metadata remains unknown and private records are account-bound.

Video. ID, title, channel ID/title, canonical URL, description, available tags/category, publication time, duration in seconds, language evidence, fetch time and availability. Missing fields remain explicitly unknown.

Channel. ID, handle when available, title, canonical URL, description and the metadata sample used for comparison.

Library collection. Authorised account binding, collection type, accessible items, retrieval time, pagination/completeness and skipped-item counts.

Ranked result. Video or channel identifier, rank, application-computed score, selection reason and evidence limitations.

Playlist proposal. Proposal ID/version, account binding, constraints, title, privacy, ordered unique video IDs, durations, total, evidence, expiry and approval state.

Execution result. Operation ID, proposal version, created playlist ID/URL, inserted and remaining items, status and safe recovery advice.

Appendix C: To Be Determined List

These open items require recorded decisions. They do not make mandatory features optional or permit silent changes to the submitted scope.

ID	Item and closure evidence	Owner and timing
TBD-01	Resolve dependency D-02: applicability of YouTube derived-data/metric restrictions to the assigned similarity and analytics features. Record the instructor/provider interpretation and permitted live scope in an ADR. Until then, distinguish fixture evidence from blocked live acceptance.	Project team with instructor/provider; before affected live use.
TBD-02	Select and document the client/version and trusted human confirmation route required by NFR-01 and FR-5.5. Verify connection, resources and exact-proposal approval.	Project team; design and client-integration stage.
TBD-03	Record the implementation runtime, dependency versions and ranking configuration under NFR-11. These selections must meet the preserved requirements and the documented local environment.	Project team; design and reproducible-build setup.